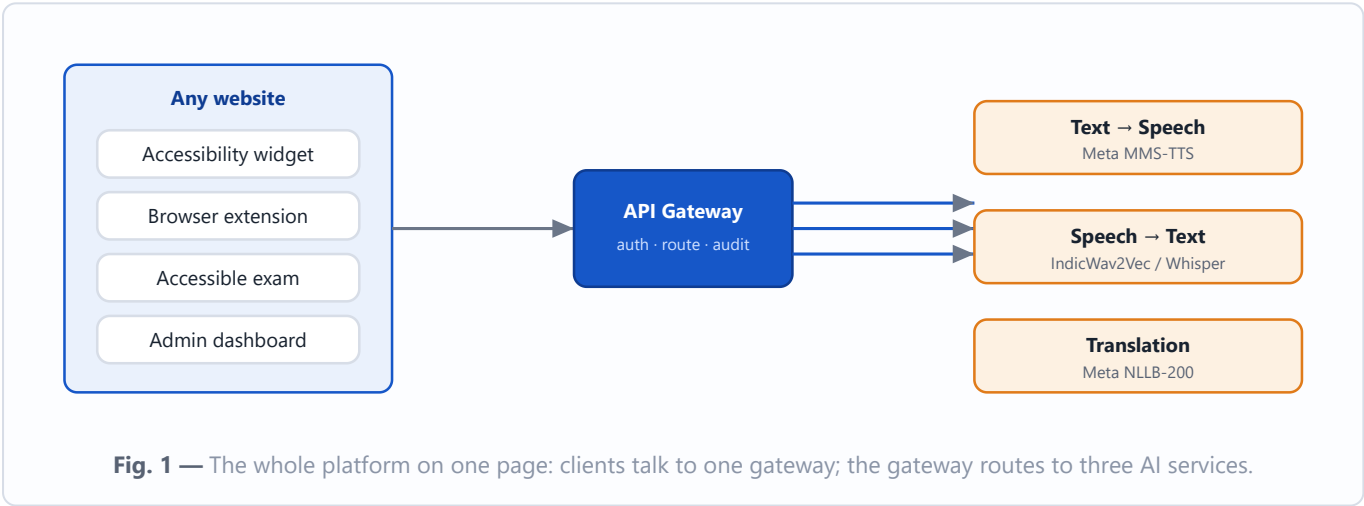


SMART ODISHA HACKATHON '25 · TECHNICAL GUIDEBOOK

Accessibility as a Service

A single, Odia-first accessibility layer that any public institution can switch on in a day — so no citizen is shut out of their own government. This book explains, in plain language and with diagrams, **what we built and exactly how it works.**

"The UPI of accessibility — Odisha first, India next."



What this guidebook covers

#	CHAPTER	WHAT YOU'LL UNDERSTAND
01	The Problem & The Big Idea	Who we serve and why accessibility should be shared infrastructure.
02	System Architecture	The gateway, the request lifecycle, and the “mock-first” safety net.
03	The Six Components	TTS, STT, Translation, Gateway, Exam, and Admin — what each does.
04	Reaching Real Websites	How the widget & extension put accessibility on any site.
05	How the AI Works	Speech and translation explained simply, with the real models.
06	Odia-First Design	Typography, dyslexia rules, and why Odia is the reference language.
07	Security, Privacy & Accessibility	DPDP data-flow, the threat model, and WCAG compliance.
08	Running & Demoing It	Two ways to run it and the two-minute judge walkthrough.
09	Pitch, Q&A & Roadmap	The pitch, hard questions answered honestly, and what’s next.

HOW TO PRINT & ASSEMBLE THIS BOOK

Open each `.html` file in **Chrome or Edge** and press **Ctrl + P**. In the print dialog set **Paper = A4**, **Margins = Default**, and turn “**Background graphics**” **ON** (so diagrams and colour boxes print). Print chapters `01` → `09` in order and clip them together behind this cover.

CHAPTER 01

The Problem & The Big Idea

Millions of citizens cannot use their own government’s websites. Today every institution is expected to fix this alone — and almost none do. We fix it once, centrally, in Odia first.

Three real people in Odisha today

The clearest way to explain the problem is through the people it shuts out. These three personas drive our entire demo.

P Priyanka
Blind, speaks only Odia. She wants her exam hall-ticket from a university portal. The page is English-only, the “Download” button is an unlabelled image. She asks her brother — every time.

A Arun
A clerk with a learning difficulty. The dense Odia text on the new e-district portal overwhelms him. He prints everything and fills it by hand, then has someone key it in.

M Meera
A Class-10 student with dyslexia. The exam’s Odia text is small and cramped, the conjuncts hard to read, with no read-aloud option. She gives up halfway.

WHY IT STAYS BROKEN

Each institution that wants to help these citizens must hire developers, buy screen-reader licences, record Odia voices, and test with disabled users. It is expensive, inconsistent, and almost never done well — so the same people keep getting left out, portal after portal.

The big idea: accessibility as shared infrastructure

India built one payments network (UPI) instead of every bank inventing its own. We apply the same idea to accessibility. Build the hard parts — Odia speech, reading, translation, accessible exams — **once, centrally,** and let any institution “plug in,” the way they plug in payments or SMS today.

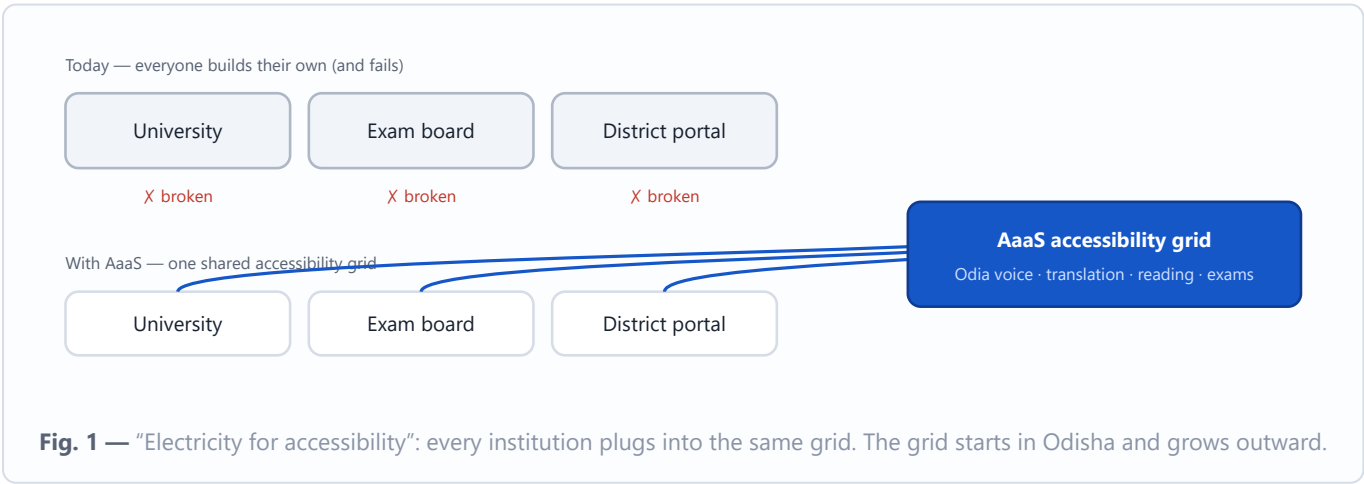


Fig. 1 — “Electricity for accessibility”: every institution plugs into the same grid. The grid starts in Odisha and grows outward.

Why Odisha, why Odia, why now

42M+

Odia speakers

21L+

persons with disabilities in
Odisha

200+

BPUT-affiliated colleges

5T

state governance mandate

Odisha has an active digital-governance push (the **5T** and **Mo Sarkar** programmes), a large population poorly served by English-first tooling, and Odia is genuinely under-served by global AI. Solving it here is both high-impact and winnable — and the policy environment (Digital India, the RPwD Act 2016, the DPDP Act 2023) rewards it.

Who actually uses it

- **End users** — blind, deaf, dyslexic, motor-impaired, elderly, and non-English speakers. They never “log in”; they just see a small accessibility button on their institution’s site.
- **Institution staff** — registrars, exam controllers, e-governance officers. They sign up, get a key, and switch features on.
- **Government & regulators** — who get compliance dashboards and adoption numbers.

THE ONE-LINE SUMMARY

A single, trustworthy, **Odia-first accessibility layer** — built for Odisha, designed for every Indian public institution — that any government body can switch on in a day. ଓଡ଼ିଶା ପାଇଁ, ଓଡ଼ିଶା ଠାରୁ। (For Odisha, from Odisha.)

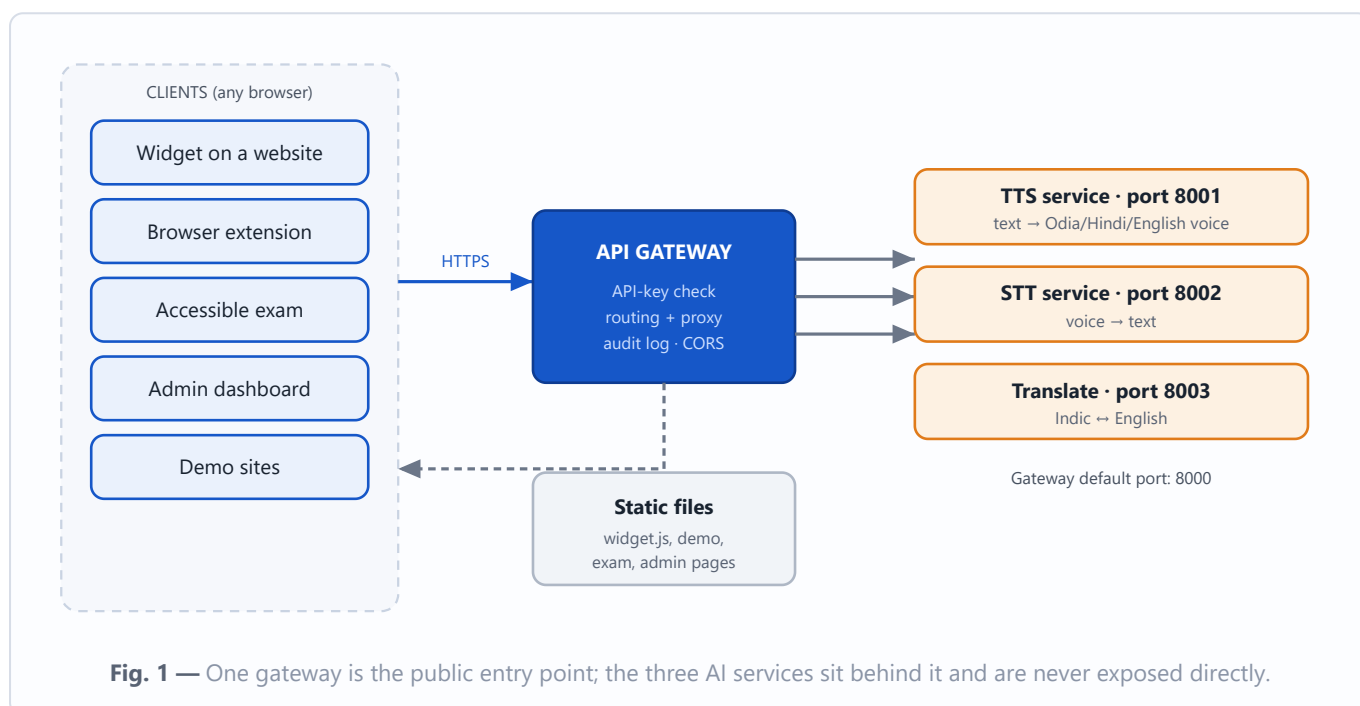
CHAPTER 02

System Architecture

One front door, three workers behind it, and a safety net that means the demo never goes dark. Here is how the pieces fit and how a single click becomes spoken Odia.

The whole system at a glance

Everything a user touches (the widget, the browser extension, the exam, the admin dashboard) talks to a single **API Gateway**. The gateway checks the request, then forwards it to whichever AI service can answer: text-to-speech, speech-to-text, or translation. Nothing else is exposed to the internet.



WHY A GATEWAY?

It gives us one place to check the institution's **API key**, attach the **tenant** (which institution is calling), **record every request** for the usage dashboard, and allow cross-site calls (CORS). Each AI service stays simple and does exactly one job.

What happens when Priyanka taps “Read this page”

Follow one real request end to end. Each numbered step is a real network call you can watch in the browser's dev tools.

- 1 **Widget reads the page.** It walks the visible text, skipping menus and decoration, and splits it into sentences.
Runs entirely in the browser — no server needed yet.
- 2 **(If needed) translate.** If the reading language differs from the page, each sentence is sent to the gateway's translation service first.

- 3 **Ask for speech.** The widget POSTs each sentence to `/tts/synthesise` on the gateway with the institution's API key.
- 4 **Gateway checks & forwards.** It validates the key, tags the tenant, records the call, and proxies it to the TTS service on port 8001.
- 5 **The model speaks.** Meta MMS-TTS turns the sentence into a `.wav` audio clip and streams it back.
- 6 **Play, sentence by sentence.** The widget plays clips as they arrive, so audio starts in about a second instead of waiting for the whole page.

The safety net: “mock-first” engines

Every AI service is built around a swappable **engine**. By default a lightweight **mock** returns sensible canned answers; a single environment-variable flip swaps in the real model. If a real model fails to load (missing weights, no internet, a driver issue), the service keeps serving with the mock — **so a live demo never crashes**.

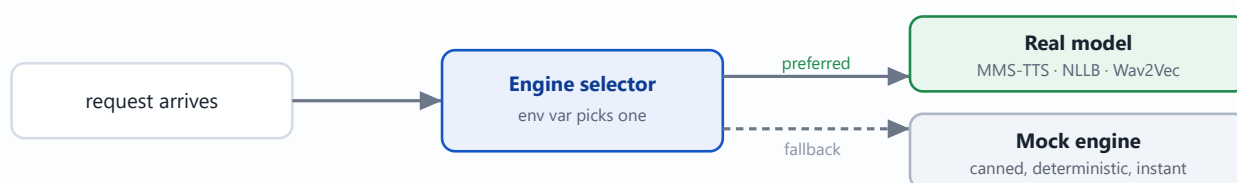


Fig. 2 — One Protocol, two backends. The demo UI comes together on mocks; real models drop in when ready.

WHAT THIS BUYS US

The full user story works on day one; every feature is testable before a heavyweight model download; and on stage, killing a model gives a clean “unavailable” message instead of a white screen. **Predictable degradation is a feature.**

Engines are chosen per service: `AAAS_STT_ENGINE`, `AAAS_TRANSLATE_ENGINE`. The TTS service always runs the real Meta MMS-TTS voice because it is small and fast on a plain CPU.

CHAPTER 03

The Six Components

The platform is six cooperating parts. Three are backend AI services; three are things people see and use. Each has one clear job, so any one can be improved or replaced without disturbing the others.

■ real engine working today ■ mock by default, real on a flag ■ user-facing surface

1 · Text-to-Speech (TTS) real

What it does: turns any text into a clear human voice in Odia, Hindi, or English. **How:** Meta's open *MMS-TTS* model (one voice per language). It normalises the text, splits long passages into sentences, runs each through the model, and returns a `.wav` clip. The Odia voice is loaded at start-up; Hindi and English load on first use. Runs comfortably on a normal CPU.

2 · Speech-to-Text (STT) mock → real

What it does: turns a short voice recording into text so people can fill forms or answer exams by speaking. **How:** the browser records up to ten seconds and sends the audio to the gateway. The real Odia engine is AI4Bharat's *IndicWav2Vec*; *Whisper* is the fallback for English and code-mixed speech. If the (access-gated) Odia model or its libraries aren't present, the service automatically serves a deterministic mock instead.

3 · Translation mock → real

What it does: converts content between Indian languages and English — so an English circular can be read in Odia. **How:** Meta's *NLLB-200* model (200 languages, runs on CPU) is the default real engine; AI4Bharat's *IndicTrans2* is an optional swap. A small built-in phrase table covers the demo if no model is loaded.

ALL THREE SPEAK THE SAME SIMPLE LANGUAGE

Every service exposes `/healthz` (am I up?), `/readyz` (is the model loaded?), and one work endpoint. They all run model inference off the main thread so the service stays responsive, and all log what they did for the audit trail.

4 · The API Gateway real

What it does: the single front door. **How:** it checks the per-institution API key, attaches the *tenant* (Utkal University, Jajpur Collectorate, BSE Odisha...), records every request into a rolling usage log, forwards the call to the right service, and also serves the widget file and the web pages — all from one place, which keeps cross-site security simple.

5 · The Accessible Exam real

What it does: a mock board-exam built for disability from the ground up. **How:** Odia questions, a "read this question aloud" button, answer-by-voice, a dyslexia-friendly font toggle, high-contrast and large-text modes, and a +50% *extra-time* timer per the RPwD Act. It reuses the same TTS and STT services through the gateway.

6 · The Admin Dashboard real

What it does: the operator's control room. **How:** it lists tenants and their API keys, shows live usage (requests, errors, latency) as a chart, and runs a one-click **WCAG accessibility scan** of any demo page using the industry-standard *axe-core* engine — producing a pass/violation report on the spot.

The endpoints each surface actually calls (all via the gateway)

COMPONENT	ENDPOINT	IN → OUT
Read-aloud	POST /tts/synthesise	text → .wav audio
Dictation	POST /stt/transcribe	audio → text
Translate	POST /translate/translate	text + languages → text
Admin · tenants	GET /admin/api/tenants	→ tenant + key list
Admin · usage	GET /admin/api/usage	→ request/error/latency stats
Admin · WCAG	axe-core (in browser)	page → pass/violation report

ONE INTEGRATION SURFACE, SIX WINS

An institution drops in **one line of code** and gets read-aloud, dictation, translation, dyslexia support, accessible exams, and WCAG monitoring — without building any of it themselves.

CHAPTER 04

Reaching Real Websites

Accessibility is useless if it can't get onto the pages people actually use. We offer two routes: the institution adds one line of code, or the citizen installs a browser extension that works *everywhere*.

Route A — the one-line widget (institution opts in)

An institution pastes a single tag into its site. The widget then loads itself and adds the floating ♿ accessibility button.

```

<!-- one line in the page's HTML -->
<script src="https://your-gateway/widget.js" data-key="aaas_live_..." defer></script>

```

Route B — the browser extension (citizen opts in)

This is the “wow” path: install once, and the same accessibility button appears on **every** website you visit — no cooperation needed from the site owner. This matters because most government sites will never add the tag themselves.

The trick that makes it safe on any page: the Shadow DOM

The widget builds its panel inside a **Shadow DOM** — a sealed bubble. The host page's styles can't leak in and break the widget, and the widget's styles can't leak out and break the page. It can sit on a bank, a news site, or a government portal without conflict.

How the extension injects the widget

A browser keeps an extension's scripts in a separate “world” from the page's scripts for security. We bridge the two using the one thing they share — the page's HTML — passing the gateway address and key through it.

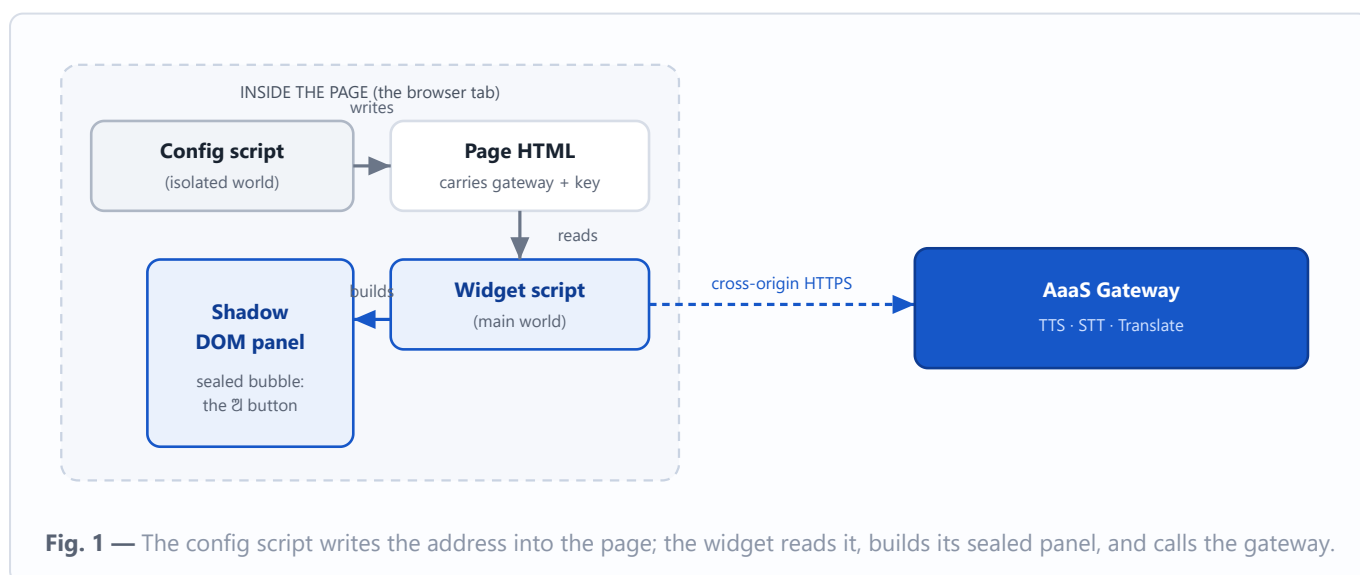


Fig. 1 — The config script writes the address into the page; the widget reads it, builds its sealed panel, and calls the gateway.

The two cross-origin hurdles (and how we cleared them)

When a widget on, say, a real university site calls our gateway, the browser applies extra safety checks. We had to satisfy two:

- **CORS** — the gateway explicitly allows calls from any origin (it only accepts an API-key header, never cookies), so a widget on any site is permitted to call it.
- **Private Network Access** — modern Chrome blocks a public website from quietly calling a local address (`127.0.0.1`) unless the server opts in. We added the `Allow-Private-Network` response so the local demo works from real pages too.

VERIFIED ON A REAL SITE

With the local services running, the widget was injected on a live third-party HTTPS page and **read the page aloud in a real Odia voice** — proving the extension path works beyond our own demo sites.

Offline option: on-device mode

The extension can run the speech models **entirely inside the browser** using WebAssembly — no server, no internet. It's slower and uses more memory, so it's optional, but it's a powerful answer to "what about poor connectivity?"

GRACEFUL FAILURE

If the gateway can't be reached, the widget shows a clear status instead of failing silently. The design principle throughout: **never leave the user in confused silence**.

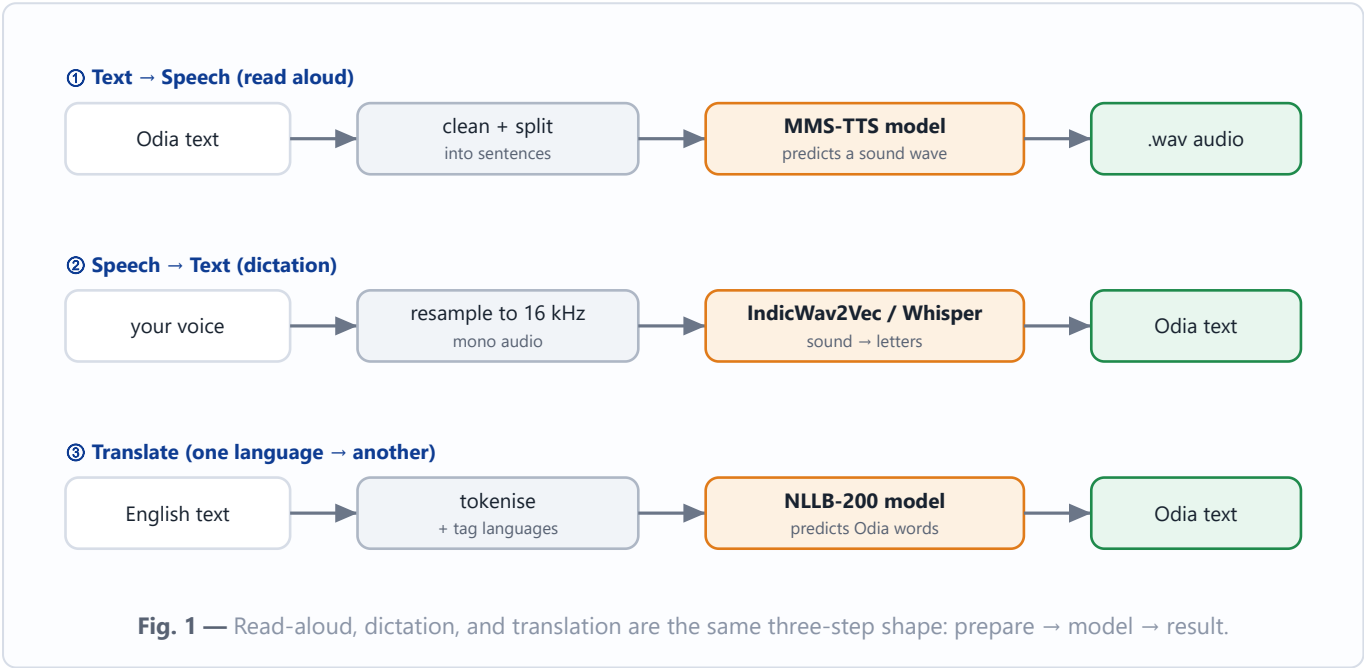
CHAPTER 05

How the AI Works

No magic — three well-understood pipelines. We do not train new models; we **integrate** the best open ones for Indian languages and wrap them in reliable, swappable services. Here is each, in plain terms.

Three pipelines, one shape

Each service is a small assembly line: clean the input, hand it to a neural model, return the output.



The models we use, and why

JOB	MODEL	WHY THIS ONE (IN ONE LINE)
Odia / Hindi / English voice	Meta MMS-TTS	Small, fast on CPU, and already has an Odia voice — most TTS systems have none.
Odia dictation	AI4Bharat IndicWav2Vec	Best published accuracy for Odia speech; far better than English-centric tools.
English / mixed dictation	Whisper	A robust, CPU-friendly fallback for English and Odia-English code-mix.
Translation	Meta NLLB-200	Covers 200 languages incl. Odia, runs on CPU, and needs no sign-in to download.

A SIMPLE WAY TO PICTURE A SPEECH MODEL

Think of MMS-TTS as someone who has heard thousands of hours of Odia and learned the exact shape of the sound wave for every letter combination. Give it letters, it draws the matching wave; play the wave, you hear Odia. Dictation is the same skill in reverse — wave in, letters out.

What the “mock” actually does

The mock isn't random. It takes a fingerprint (a hash) of the input and always returns the **same** sensible answer for the same input — for example, a fixed set of Odia demo phrases for dictation, or a curated phrase table for translation. That determinism makes automated tests reliable and the demo repeatable, even with no model present.

Honest about quality

WE SAY WHAT WORKS AND WHAT DOESN'T

Standard Odia (Bhubaneswar/Cuttack) dictation is good; rural dialects like Sambalpuri and Kosali are not yet benchmarked and are weaker. Translation is solid for common sentences. We publish accuracy rather than pretend — judges trust honesty more than inflated claims.

Every model here is open and Indic-capable. The one licensing caveat: the MMS-TTS voice is licensed for non-commercial use, so a real pilot would swap to a commercially-cleared Odia voice — a change isolated to one file.

CHAPTER 06

Odia-First Design

“Odia-first” is not a slogan. It means Odia is the **reference language** — every feature is built and validated for Odia before any other language ships. That forces decisions most tools get wrong.

The font travels with the widget

Many Windows machines render Odia conjuncts poorly (the default “Kalinga” font breaks them). So we **bundle Noto Sans Oriya inside the widget itself** — embedded directly in the code. Wherever the widget loads, Odia renders correctly, even offline, even on a fresh laptop. Odia also gets a more generous default line-height (1.7) than Latin text (1.5).

The dyslexia insight everyone else misses

For English, dyslexia tools add *letter-spacing* and switch to a font like OpenDyslexic. **Doing that to Odia is actively harmful.** Odia letters join into conjuncts — single shapes made of multiple letters. Adding letter-spacing rips those shapes apart, making text *harder* to read. OpenDyslexic doesn’t even support the script.

So our Odia dyslexia mode does the opposite: it keeps Noto Sans Oriya, sets letter-spacing to **zero**, and instead increases **word-spacing** and pushes line-height to ~1.9. More breathing room between words and lines, without ever breaking a conjunct.

✗ NAÏVE “DYSLEXIA” STYLING (LETTER-SPACED)

ଓଡ଼ିଶା ମାଧ୍ୟମିକ ଶିକ୍ଷା ପରିଷଦ

Letter-spacing pulls conjuncts apart — the base letter and its marks drift, harder to read.

✓ ODIA-AWARE DYSLEXIA MODE (OURS)

ଓଡ଼ିଶା ମାଧ୍ୟମିକ ଶିକ୍ଷା ପରିଷଦ

Zero letter-spacing, more word-spacing & line-height — calmer, conjuncts intact.

(Both lines read “Board of Secondary Education, Odisha.” If your printer lacks an Odia font the shapes may differ, but the spacing principle still shows.)

Cultural correctness, not just translation

- **Formal address.** The interface uses the respectful ଆପଣ (“you”), as a government service should.
- **Odia numerals.** The system understands Odia digits (୨୦୦୫ = 2005), not only Western ones.
- **Real institutions.** Demo sites mirror genuine Odisha bodies — Utkal University, BSE Odisha, Jajpur Collectorate — with real scheme names like the Madhu Babu Pension Yojana.

Language coverage

Today

Odia, Hindi, English for voice and translation — the three the demo speaks end to end.

Next

Bengali, Tamil, Telugu and more — the translation model already covers 200 languages.

Research

Sambalpuri / Kosali / Desia dialect robustness for Odia speech — openly flagged as not-yet-solved.

THE POSITIONING, IN ONE SENTENCE

“Odia as a **reference language**, not a translation target.” We design for Odia first and let the rest follow — the opposite of building in English and bolting Odia on at the end.

CHAPTER 07

Security, Privacy & Accessibility

Accessibility tools touch sensitive data — voices, disability records, exam papers. Government won't adopt a black box. So trust is designed in: minimal data, Indian residency, and accessibility treated as correctness.

Privacy under the DPDP Act 2023

India's data-protection law shapes our defaults. The guiding rule is simple: **collect the least, keep it the shortest time, and never let it leave the country.**

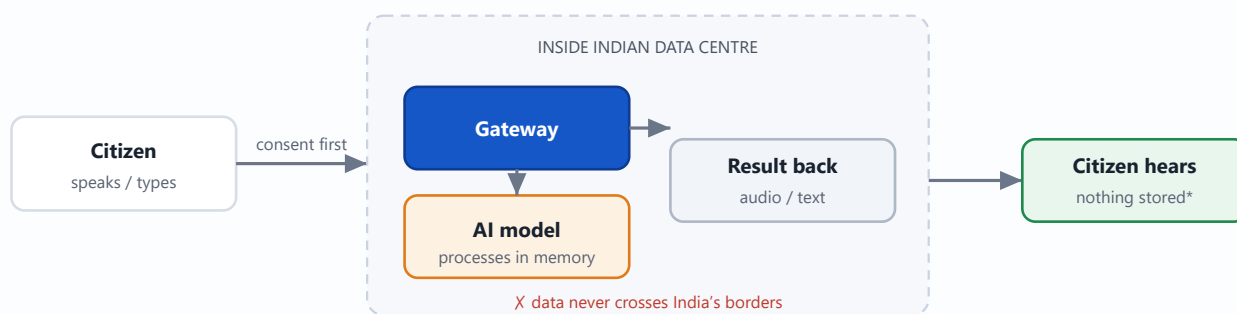


Fig. 1 — By default the voice clip is processed in memory and discarded — *unless the user explicitly opts in to help improve models.

Consent-first

The microphone is never on by default. Access is explicit, per session, and visible.

Minimal & short-lived

Audio isn't stored beyond answering the request. Sensitive records have strict retention limits.

Data stays in India

All processing runs in Indian data centres — designed for an on-prem install at OCAC, Bhubaneswar.

User control

A citizen can export or delete everything the platform knows about them, from one page.

Security model (STRIDE) — the highlights

- **Encrypted everywhere.** TLS from browser to gateway; secrets live in a key store, never in plain files in production.
- **API keys are hashed.** We store only a one-way fingerprint of each key, never the key itself, and show it only once when minted.
- **Locked-down services.** Each AI service runs as an unprivileged user; audio is deleted within the hour.
- **Supply-chain checks.** Automated dependency scanning and a software bill-of-materials on every build.

HONEST SCOPE FOR THE HACKATHON BUILD

The demo uses simple in-memory tenants and a seed key — production would add full sign-in (Keycloak/DigiLocker), a database, and rate-limits. We document this gap rather than hide it.

Accessibility as correctness, not decoration

Our own surfaces are held to **WCAG 2.2 AA** (the recognised standard), and we go beyond it where it matters: body-text contrast at the stricter **AAA** level, touch targets of at least 44×44 pixels, and full respect for a user's "reduce motion" setting.

WE MEASURE IT, LIVE

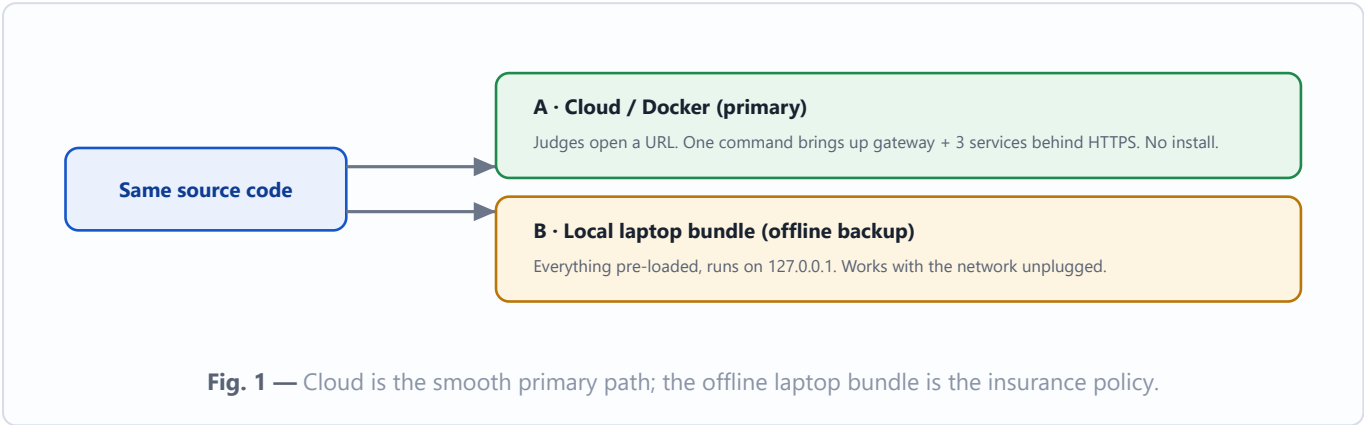
The admin dashboard runs **axe-core** — the same engine professional auditors use — against any page and returns a pass/violation report in seconds. In testing it scored the Jajpur demo site at **20 passes, 0 violations**. Accessibility you can put a number on.

A code change that breaks accessibility fails the automated checks and is blocked from merging — the same way a broken test would be. That is what "accessibility is a correctness concern" means in practice.

CHAPTER 08

Running & Demoing It

Two independent ways to run, so a dead Wi-Fi or a locked-down laptop never kills the demo. Both run the exact same code.



Running it locally (developer / offline)

Four small web services run on four ports. Start them, open the browser, done.

SERVICE	ADDRESS	PURPOSE
Gateway	127.0.0.1:8000	Front door + all the web pages
TTS	127.0.0.1:8001	Odia / Hindi / English voice
STT	127.0.0.1:8002	Voice → text
Translate	127.0.0.1:8003	Indic ↔ English

```
# one-time setup (downloads the Python libraries + models)
SETUP-FRIEND.bat
# every time you want the demo
RUN-DEMO.bat      # opens 4 windows, then the browser at /demo/
```

Then visit `http://127.0.0.1:8000/demo/` for the landing page with the three Odisha demo sites, the exam, and the admin dashboard.

The two-minute judge walkthrough

“Priya is a blind student in Jajpur applying to Utkal University.” Every step degrades gracefully if something misbehaves.

- 1 Open the Utkal University demo.** The button is in the corner. Press it → “Read this page” → the page reads aloud in Odia.
- 2 Switch to English & translate.** Flip the language; the page re-reads in English for her English-speaking helper. Translation runs through our own gateway.

- 3 **Fill a field by voice.** Tap the mic, speak a name in Odia, watch it become text in the form.
- 4 **Open the exam.** It loads with a dyslexia-friendly font, high contrast, and a +50% extra-time banner; each question can be read aloud and answered by voice.
- 5 **Open the admin dashboard.** Show the tenant list, the live usage chart (Priya's session is visible), and a one-click WCAG scan returning an AA report.
- 6 **Land the message.** "One integration surface — TTS, dictation, translation, and WCAG monitoring. Six compliance wins from one line of code."

If something breaks (rehearsed fallbacks)

IF...	THEN...
Venue Wi-Fi dies	Switch to the offline laptop bundle (127.0.0.1).
A model misbehaves	The mock answers instead — the flow keeps moving.
The mic fails	Type the Odia input; the rest of the story is unaffected.
The whole laptop dies	Hand over this guidebook and give the 60-second pitch (Chapter 09).

DESIGN THE DEMO AROUND WHAT CANNOT FAIL

The voice, the page-flip to Odia, and the dashboard all work without the riskiest AI being perfect. We lead with the parts that are rock-solid and keep a recorded run ready as the final safety net.

CHAPTER 09

Pitch, Q&A & Roadmap

Everything you need to defend the project in a hallway, at the booth, or in front of the panel — the hooks, the numbers, the hard questions, and an honest map of what's done versus what's next.

The pitch, two lengths

15 SECONDS

"Every government website makes citizens with disabilities ask someone else for help. We built one shared, Odia-first accessibility layer any institution can switch on in a day — the UPI of accessibility."

60 SECONDS

"Odisha has 42 million Odia speakers and 21 lakh persons with disabilities, yet government portals are English-first and unusable for them. Today every institution is expected to fix this alone, and almost none do. Our insight: accessibility is **infrastructure**, not 200 separate projects. So we built a single platform — Odia voice, dictation, translation, a dyslexia-aware reading mode, accessible exams, and a WCAG compliance dashboard — behind one API. An institution adds one line of code and its users get all of it. It's open-source, runs on-prem inside India for DPDP compliance, and it's working today. We're asking you to help us pilot it at one Odisha institution."

Numbers to know cold

42M+

Odia speakers

21L+

PwD in Odisha

6

components, 1 API

0WCAG violations on the
demo site

Hard questions, honest answers

"Isn't this just an accessibility overlay like AccessiBe?"

No. Overlays bolt onto broken HTML at run-time and the disability community criticises them. We provide an *opt-in assistive layer* the user controls **and** a WCAG audit tool that helps the institution fix the root cause. We make sites better, we don't paper over them.

"You didn't train the AI models — where's the innovation?"

Correct, and deliberate. The models (MMS-TTS, NLLB, IndicWav2Vec) are open and excellent; reinventing them would be foolish. The hard, valuable work is the **integration**: a reliable multi-tenant platform, the Odia-specific design rules, graceful failure, DPDP controls, and one-line adoption. UPI didn't invent banks — it connected them.

“How do I know it actually works for Odia?”

Watch the demo: it reads real Odia aloud and translates a live page into correct Odia through our own engine. We’re also honest about limits — standard Odia is strong; rural dialects are future work, not a solved claim.

“It’s a hackathon project — is the rest vapourware?”

Check the repository: a real threat model, a DPDP data-flow map, a machine-enforced WCAG checklist, CI, and an SBOM are all there. The six components run today. We show what’s built and label what’s roadmap.

What’s built today vs what’s next

- Done · Foundations & working slice**
Design system, CI, threat model, DPDP map, WCAG harness — plus all six components running: TTS, STT, translation, gateway, exam, admin.
- Done · Real websites**
Browser extension + widget verified injecting and reading aloud on live third-party sites; translation routed through our own gateway.
- Next · Pilot hardening**
Real sign-in (Keycloak/DigiLocker), a database for tenants & keys, rate-limits, and a commercially-licensed Odia voice.
- Next · Language & assistance breadth**
More Indian languages, a screen-reader overlay, sign-language avatar, and a public accuracy benchmark per language.
- Later · Scale & certification**
Reference deployment at OCAC Bhubaneswar, ISO 27001 / CERT-In, and a state-wide onboarding playbook.

The tech stack, briefly

LAYER	CHOICE	WHY
Backend services	Python + FastAPI	The natural home for AI/ML, fast to build, easy to read.
Widget	Plain JS + Shadow DOM	Tiny, dependency-free, and conflict-proof on any host page.
Voice / Speech / Translate	MMS-TTS · IndicWav2Vec/Whisper · NLLB-200	Best open, Indic-capable, CPU-friendly models — no vendor lock-in.
Accessibility audit	axe-core	The industry-standard WCAG engine; runs client-side, works offline.
Packaging	Docker (cloud) + offline bundle	Two independent, identical demo paths.

Weaknesses we state up front

Naming these earns more trust than hiding them: no professionally-trained accessibility auditor on the core team yet; no disabled Odia-speaking user testing done so far; no signed pilot institution (targets identified);

and rural-dialect speech is unproven. Each is openly sequenced into the roadmap.

"Accessibility is not a feature; it is a part of citizenship."

Team SUBARNAREKHA · Accessibility as a Service · Apache-2.0 · Smart Odisha Hackathon '25